1 一様重力場中の単ふりこ

Python (SciPy, NumPy 等) を使って一様重力場中の単ふりこの問題を数値的に解きます。

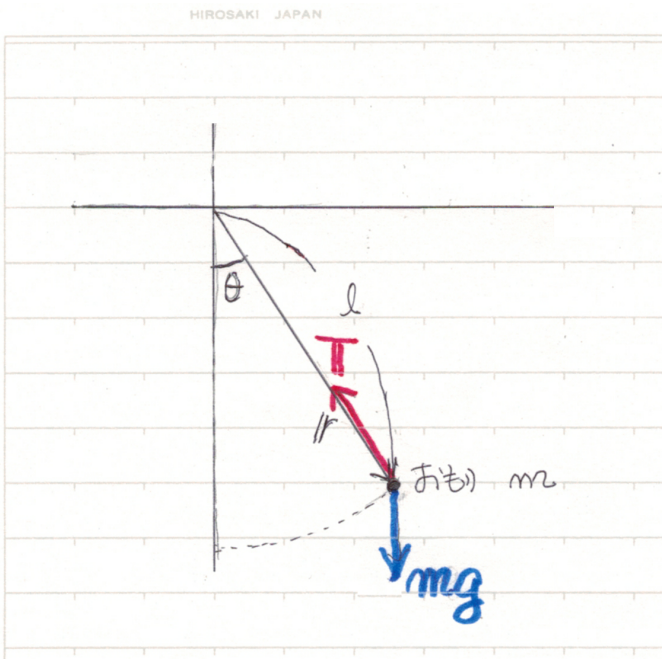
振幅が小さい時には単振動になり，振り子の周期は振幅に依存しないことが知られていますが，振幅が大きいときの（「十分小さい」という近似が成り立たない場合の）ふりこの周期は，振幅にどう依存するでしょうか？

1.1 単ふりこの運動方程式

単ふりこのひもの長さを ℓ・重力加速度の大きさを g・鉛直下向きからの振れ角を θ とすると，単ふりこの運動方程式は

$$\frac{d^2\theta}{dt^2} = -\frac{g}{\ell}\sin\theta$$

となることを示す。



1.1.1 運動方程式

（以下・Jupyter Notebook の数式表示では，3次元ベクトルを `boldsymbol` $\boldsymbol{r}$ のように表すと普通の r と区別がつきにくい懸念があるので，$\vec{r}$ のように表すことにする。）

単ふりこの先端のおもりは，ひもの張力 $\vec{T}$ と重力 $m\vec{g}$ を受ける。鉛直下向きを $+z$ 方向とすると $\vec{g} = (0, 0, g)$ は重力加速度ベクトル, g は重力加速度の大きさである。

したがって，運動方程式は

$$m\frac{d^2\vec{r}}{dt^2} = m\vec{g} + \vec{T}$$

1.1.2 保存量（角運動量の鉛直成分）

この式の両辺に位置ベクトル $\vec{r}$ の外積をかける。$\vec{T} \propto -\vec{r}$ であることに注意すると，$\vec{r} \times \vec{T} = \vec{0}$ であるから，運動方程式は以下のようになる。

$$\frac{d}{dt}\left(m\vec{r} \times \frac{d\vec{r}}{dt}\right) = m\vec{r} \times \vec{g}$$

この両辺に一定のベクトルである重力加速度ベクトル $\vec{g}$ の内積をかけてやると，

$$\frac{d}{dt}\left\{\vec{g} \cdot \left(m\vec{r} \times \frac{d\vec{r}}{dt}\right)\right\} = 0$$

この式は，角運動量 $\vec{L} \equiv m\vec{r} \times \dfrac{d\vec{r}}{dt}$ の鉛直成分・つまり重力加速度ベクトルにそった成分が保存することを意味する。

1.1.3 運動が平面内に限られる理由

この式を成分で具体的に書き下すと（両辺を mg で割ったのち）

$$\frac{d}{dt}\left(x\frac{dy}{dt} - y\frac{dx}{dt}\right) = \frac{d}{dt}\left\{x^2\frac{d}{dt}\left(\frac{y}{x}\right)\right\} = 0$$

極座標

$$\vec{r} = (x, y, z) = (\ell\sin\theta\cos\varphi, \ell\sin\theta\sin\varphi, \ell\cos\theta)$$

を使って書くと，

$$\frac{d}{dt}\left\{x^2\frac{d}{dt}\left(\frac{y}{x}\right)\right\} = \ell^2\frac{d}{dt}\left(\sin^2\theta\frac{d\varphi}{dt}\right) = 0$$

微分してゼロということは，中身が一定であることだから，

$$\sin^2\theta\frac{d\varphi}{dt} = \mathrm{const.} \equiv l_z$$

ここで，初期条件として $t = 0$ で $\dfrac{d\varphi}{dt} = 0$ とすると，$l_z = 0$ となり常に $\dfrac{d\varphi}{dt} = 0$，つまり $\varphi = \mathrm{const.}$ でありつづけることが示される。

したがって，単ふりこの運動が（慣性系では）一定の平面内に限られるのは，以下の理由による：

- 角運動量の鉛直成分 (重力加速度ベクトルに平行な成分) は保存される。（上記の場合は，重力加速度ベクトルの向きは z 方向なので，角運動量の z 成分 $L_z = m\ell^2 l_z$ が一定である。）
- 初期条件として $L_z = 0$ とすると（上記の場合は $\dfrac{d\varphi}{dt} = 0$ に相当。）・ずーっと $L_z = 0$ でありつづける。（上記の場合は $\dfrac{d\varphi}{dt} = 0$，つまり $\varphi = \mathrm{const.}$）
- したがって，運動は φ が一定の平面内に限られる。

すなわち，初期条件として，$y = 0, \dfrac{dy}{dt} = 0$ とすると，以後の 単ふりこの運動は $y = 0$ の xz 平面内に限られることがわかる。

1.1.4 最終的な運動方程式

そこでこれ以後は，$\vec{r} = (x, y, z) = (\ell\sin\theta, 0, \ell\cos\theta)$ とおく。

すると，$\vec{r}$ と外積をとった運動方程式の残りの成分のうち，x 成分は $0=0$ となり， y 成分は以下のようになる。

$$\begin{aligned}\frac{d}{dt}\left(z\frac{dx}{dt}-x\frac{dz}{dt}\right)&=-xg\\ \frac{d}{dt}\left\{z^2\frac{d}{dt}\left(\frac{x}{z}\right)\right\}&=-xg\\ \ell^2\frac{d}{dt}\left\{\cos^2\theta\frac{d}{dt}\left(\frac{\sin\theta}{\cos\theta}\right)\right\}&=-\ell g\sin\theta\\ \ell^2\frac{d^2\theta}{dt^2}&==-\ell g\sin\theta\end{aligned}$$

であるから，運動方程式は最終的に

$$\frac{d^2\theta}{dt^2}=-\frac{g}{\ell}\sin\theta$$

となる。

2 振幅が小さい場合

$|\theta|\ll 1$ の場合には，$\sin\theta\simeq\theta$ と近似することにより，運動方程式は

$$\frac{d^2\theta}{dt^2}\simeq-\frac{g}{\ell}\theta$$

となり，これは単振動となる。

2.1 単振動の周期

単振動の周期 T_0 は

$$T_0=2\pi\sqrt{\frac{\ell}{g}}$$

となり，これは初期条件で与えられる振れ幅 (振幅) θ_0 に依存しない。

3 振幅が大きい場合の周期は？

振幅が大きい，つまり $\sin\theta \simeq \theta$ と近似できない場合は，単振動ではない運動をするはずだから，ふりこの周期は一般に振幅に依存するのではないかと考えられる。

そこで，振幅 θ_0 をたとえば 10° から 90° まで 10° きざみで大きくしていった場合・ふりこの周期はどうなるか？　というのが問題。

3.1 無次元化された運動方程式

解析的に解けない場合は，数値的に解くことになる。そんなとき，解くべき方程式を無次元化しておくとよい。

ここでは，単振動の場合の周期 T_0 の2乗を運動方程式の両辺にかける。 すると，右辺は

$$-T_0^2 \times \frac{g}{\ell}\sin\theta = -4\pi^2\sin\theta$$

となる。また，規格化された時間 τ を

$$\tau \equiv \frac{t}{T_0}$$

のように定義すると

$$T_0^2\frac{d^2}{dt^2} = \frac{d^2}{d\tau^2}$$

となるので，無次元化された運動方程式は以下のようになる。

$$\frac{d^2\theta}{d\tau^2} = -4\pi^2\sin\theta$$

まず，この2階常微分方程式を数値的に解いてみる。

4 運動方程式の数値解

4.1 scipy.integrate.solve_ivp()

常微分方程式系を数値的に解く `scipy.integrate.solve_ivp()` を使って運動方程式の数値解を求めます。

In [1]:

```
from scipy.integrate import solve_ivp
import numpy as np
```

`scipy.integrate.solve_ivp()` を使う。マニュアルは,

- scipy.integrate.solve_ivp - SciPy v1.6.0 Reference Guide

解くべき2階常微分方程式

$$\frac{d^2\theta}{d\tau^2} = -4\pi^2 \sin\theta$$

を以下のように，1階連立常微分方程式の形にして， `scipy.integrate.solve_ivp()` を利用しやすくする。

`scipy.integrate.solve_ivp()` は

$$\frac{d\boldsymbol{y}}{dt} = \boldsymbol{F}(t, \boldsymbol{y})$$
$$\boldsymbol{y} = [y_0, y_1, \ldots]$$

$$\boldsymbol{F}(t, \boldsymbol{y}) = [F_0(t, \boldsymbol{y}), F_1(t, \boldsymbol{y}), \ldots]$$

の形を解く。

$x = \theta, v = \dot{\theta}$ とし， `y = [y[0], y[1]] = [x, v]` 。また,

$$\frac{dx}{dt} = F_0(t, \boldsymbol{y}) = \dot{\theta} = v$$
$$\frac{dv}{dt} = F_1(t, \boldsymbol{y}) = -4\pi^2 \sin x$$

であるから,

In [2]:

```
def F(t, y):
# solve.ivp() に渡す関数の引数の順番に注意。t が先
    x = y[0]
    v = y[1]
    F0 = v
    F1 = -4*np.pi**2*np.sin(x)
    return [F0, F1]
```

In [3]:

```
# 初期条件
y_ini = [10 * np.pi/180, 0]

t_ini = 0
t_end = 2
t_span = [t_ini, t_end]

# 数値解を計算するポイント
# ここでは t_span 区間を N 分割する
N = 100
t_list = np.linspace(t_ini, t_end, N+1)

ansivp1 = solve_ivp(F, t_span, y_ini,
                    t_eval = t_list,
                    rtol = 1.e-12,
                    atol = 1.e-12)

# odeint では刻み幅と精度は連動しているが,
# solve_ivp では精度は rtol, atol で指定
```

計算の精度を確認するため， `rtol` と `atol` の値を変えてみます。

In [4]:

```
ansivp2 = solve_ivp(F, t_span, y_ini,
                    t_eval = t_list,
                    rtol = 1.e-13,
                    atol = 1.e-13)
```

計算結果は `y` に格納されます。 `y[0]` すなわち θ の最後 $\tau = 2$ での値を比べてみます。

In [5]:

```
print(ansivp1.y[0][-1])
print(ansivp2.y[0][-1])
```

```
0.17448305666770444
0.1744830566692668
```

`rtol = 1.e-12, atol = 1.e-12` で小数点以下11桁くらいの精度があることがわかりました。これで計算をつづけてみます。

In [6]:

```
# 初期条件
y_ini = [10 * np.pi/180, 0]
ans10 = solve_ivp(F, t_span, y_ini,
                  t_eval = t_list,
                  rtol = 1.e-12,
                  atol = 1.e-12)
# 初期条件
y_ini = [60 * np.pi/180, 0]
ans60 = solve_ivp(F, t_span, y_ini,
                  t_eval = t_list,
                  rtol = 1.e-12,
                  atol = 1.e-12)
```

4.2 plt.plot()

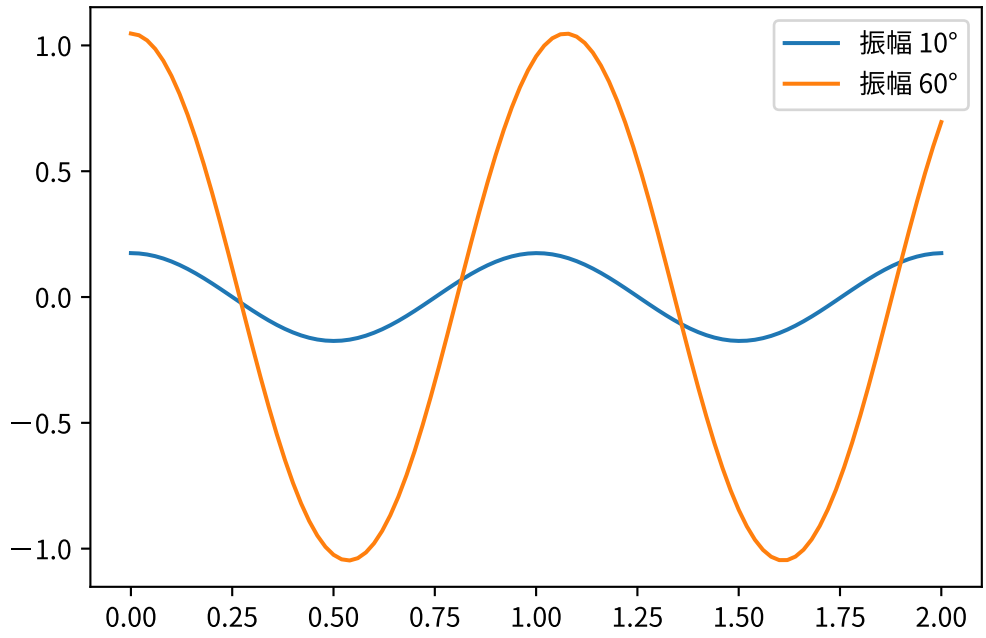
振幅 10° の場合と 60° の場合の数値解をプロットします。

In [7]:

```
import matplotlib.pyplot as plt
# 以下はグラフを SVG で Notebook にインライン表示させる設定
from IPython.display import set_matplotlib_formats
%matplotlib inline
set_matplotlib_formats('svg')
```

In [8]:

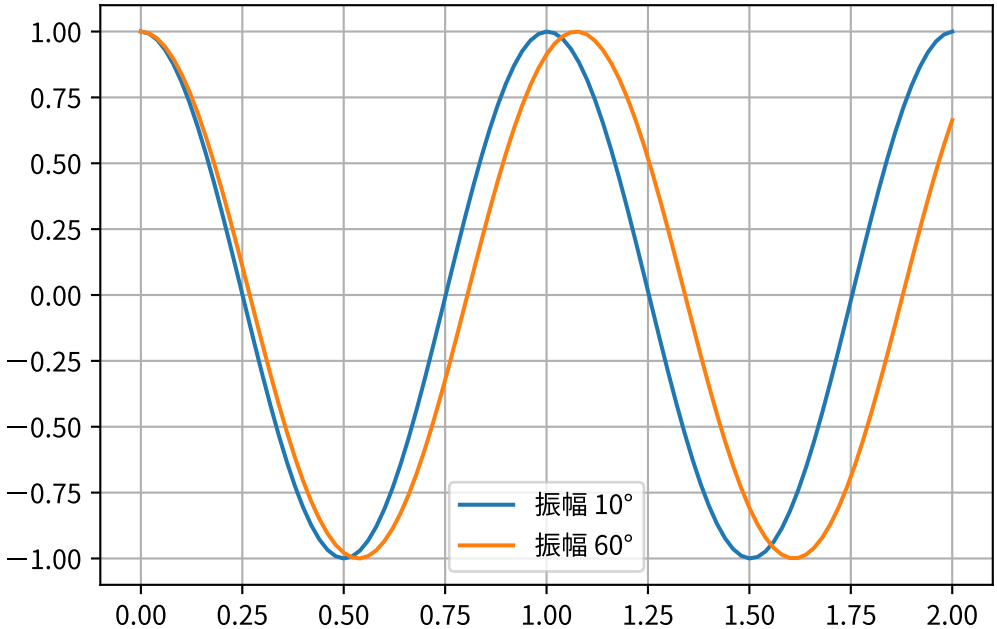
```
plt.plot(t_list, ans10.y[0], label="振幅 10°");
plt.plot(t_list, ans60.y[0], label="振幅 60°");
plt.legend();
```



判断しやすいように，初期条件の振幅で規格化した数値解をプロットします。

In [9]:

```
plt.plot(t_list, ans10.y[0]/ans10.y[0][0], label="振幅 10°");
plt.plot(t_list, ans60.y[0]/ans60.y[0][0], label="振幅 60°");
plt.grid();
plt.legend();
```



上のグラフを見る限りでは，振幅が60°と大きくなると，周期がのびているように見えます。

もう少し定量的に調べてみます。

数値計算の答えのうち，$\dot{\theta}$ の数値解は `y[1]` に格納されています。この数値解をみて，θ の値が増加から減少に転じる時刻・つまり $\dot{\theta}$ の値が正から負に転じる時刻を以下のようにして調べてみます。

In [10]:

```
# 振幅10°の場合の y[1] が正から負に転じる時刻を調べる

for i in range(len(ans10.y[1])-1):
    if ans10.y[1][i] >=0 and ans10.y[1][i+1] <=0:
        print(t_list[i])
```

```
0.0
1.0
```

振幅が $10°$ の場合は，

$$\tau = \frac{t}{T_0} = 1.0, \quad \therefore\ t = T_0$$

となり，単振動の場合の周期 T_0 と（ほぼ）同じであることがわかります。

In [11]:

```
# 振幅60°の場合の y[1] が正から負に転じる時刻を調べる

for i in range(len(ans60.y[1])-1):
    if ans60.y[1][i] >=0 and ans60.y[1][i+1] <=0:
        print(t_list[i])
```

```
0.0
1.06
```

4.3 問題

無次元化された運動方程式を数値的に解き，振幅 θ_0 を 10° から 90° まで 10° きざみで大きくしていった場合・ふりこの周期はどのように変化するかを示せ。

上記の数値計算では，

```
# 数値解を計算するポイント
# ここでは t_span 区間を N 分割する
N = 100
t_list = np.linspace(t_ini, t_end, N+1)
```

としているが，分割数をもう少し増やして，$\dot{\theta}$ の値が正から負に転じる時刻をもう少し詳しく調べてみてください。

5 保存量（エネルギー保存則）

無次元化された運動方程式

$$\frac{d^2\theta}{d\tau^2} = -4\pi^2 \sin\theta$$

の両辺に $\dfrac{d\theta}{d\tau}$ をかけて整理すると，

$$\frac{d}{d\tau}\left\{\frac{1}{2}\left(\frac{d\theta}{d\tau}\right)^2 - 4\pi^2\cos\theta\right\} = 0$$

つまり，

$$\frac{1}{2}\left(\frac{d\theta}{d\tau}\right)^2 - 4\pi^2\cos\theta = \mathrm{const.} \equiv \varepsilon$$

という保存量が存在することがわかる。（これはエネルギー保存則です。）

$t = 0$ で $\theta = \theta_0, \dfrac{d\theta}{d\tau} = 0$ という初期条件を課すと，定数 ε の値は以下のように定まる。

$$\varepsilon = -4\pi^2\cos\theta_0$$

つまり，

$$\frac{d\theta}{d\tau} = \pm 2\pi\sqrt{2(\cos\theta - \cos\theta_0)}$$

これは変数分離形の微分方程式であり，θ が 0 から θ_0 までの時間の4倍が周期 $\bar{T}$ になることから，

$$\frac{1}{2\pi}\int_0^{\theta_0}\frac{d\theta}{\sqrt{2(\cos\theta - \cos\theta_0)}} = \int_0^{\bar{T}/4} d\tau$$

すなわち

$$\bar{T} = \frac{2}{\pi}\int_0^{\theta_0}\frac{d\theta}{\sqrt{2(\cos\theta - \cos\theta_0)}}$$

この積分は解析的には解けないので数値積分すればいいと思いますが，このままの形だと，積分区間の上限で被積分関数の分母がゼロになってしまい，つまりは被積分関数が発散してしまうのでちょっと困る。なので以下のようにうまく変数変換することで，数値積分が可能な形にします。

5.1 第1種完全楕円積分

ここでは，上記の積分が変数変換によって，第1種完全楕円積分

$$K(k) \equiv \int_0^{\pi/2} \frac{dt}{\sqrt{1-k^2\sin^2 t}}$$

で書けることを示す。この形だと，$k^2 < 1$ なら積分区間内で被積分関数が発散することもなく，数値積分に困ることもない。

まず，分母は半角の公式を使って

$$\begin{aligned}\sqrt{2(\cos\theta - \cos\theta_0)} &= \sqrt{2\left\{\left(1-2\sin^2\frac{\theta}{2}\right)-\left(1-2\sin^2\frac{\theta_0}{2}\right)\right\}} \\ &= 2\sqrt{\sin^2\frac{\theta_0}{2}-\sin^2\frac{\theta}{2}}\end{aligned}$$

ここでさらに

$$\sin\frac{\theta_0}{2} \equiv k, \quad \sin\frac{\theta}{2} \equiv k\sin t$$

とおくと，

$$\begin{aligned}2\sqrt{\sin^2\frac{\theta_0}{2}-\sin^2\frac{\theta}{2}} &= 2k\sqrt{1-\sin^2 t} \\ &= 2k\cos t\end{aligned}$$

一方・分子は

$$\sin\frac{\theta}{2} \equiv k\sin t$$

の微分をとることにより，

$$\cos\frac{\theta}{2}\frac{d\theta}{2} = k\cos t\,dt$$

$$\therefore\ d\theta = \frac{2k\cos t}{\cos\frac{\theta}{2}}dt = \frac{2k\cos t}{\sqrt{1-\sin^2\frac{\theta}{2}}}dt = \frac{2k\cos t}{\sqrt{1-k^2\sin^2 t}}dt$$

したがって，

$$\begin{aligned}\int_0^{\theta_0}\frac{1}{\sqrt{2(\cos\theta-\cos\theta_0)}}d\theta &= \int_0^{\pi/2}\frac{1}{2k\cos t}\frac{2k\cos t}{\sqrt{1-k^2\sin^2 t}}dt \\ &= \int_0^{\pi/2}\frac{dt}{\sqrt{1-k^2\sin^2 t}} \\ &= K(k)\end{aligned}$$

まとめ：

単ふりこの規格化された周期 $\bar{T}$ は第1種完全楕円積分 $K(k)$ を使って

$$\bar{T} = \frac{T}{T_0} = \frac{2}{\pi}K(k)$$

と書ける。

5.2 scipy.special.ellipk()

ありがたいことに，SciPy では第1種完全楕円積分が使える。マニュアルは以下を参照：

- scipy.special.ellipk - SciPy v1.6.0 Reference Guide

In [12]:

```
from scipy.special import ellipk
```

θ_0 が 60° のときの周期を求めるには，

$$k = \sin\frac{\theta_0}{2} = \sin\frac{60\times\pi}{180\times 2}$$

の値を使って...

In [13]:

```
k = np.sin(60*np.pi/(180*2))
```

$$\frac{2}{\pi}K(k^2)$$

に入れればよい。 `scipy.special.ellipk()` の定義は以下のようになっていることに注意：

$$K(m) \equiv \int_0^{\pi/2} \left[1 - m\sin^2 t\right]^{-1/2} dt$$

In [14]:

```
2/np.pi * ellipk(k**2)
```

Out[14]:

```
1.0731820071493645
```

5.3 問題

第1種完全楕円積分を使って書かれたふりこの周期の式を使い，振幅 θ_0 を 10° から 90° まで 10° きざみで大きくしていった場合・ふりこの周期はどのように変化するかを示せ。

また，運動方程式を数値的に解いて得られた結果と比較せよ。

6 参考：第1種完全楕円積分の数値積分

第1種完全楕円積分

$$K(m) \equiv \int_0^{\pi/2} \left[1 - m\sin^2 t\right]^{-1/2} dt$$

を数値積分で求めてみる。

6.1 scipy.integrate.quad()

In [15]:

```
from scipy.integrate import quad
import numpy as np

def k_f(x, m):
    return 1/np.sqrt(1 - m*np.sin(x)*np.sin(x))

def Knum(m):
    return quad(k_f, 0, np.pi/2, args=m)[0]
```

In [16]:

```
print(2/np.pi * ellipk(k**2))
print(2/np.pi * Knum(k**2))
```

```
1.0731820071493645
1.0731820071493645
```

ということから，第1種完全楕円積分は（おそらく内部的には同じことをしているのであろうと想像できますが）組み込み関数 `scipy.special.ellipk()` でも，数値積分してくれる関数 `scipy.integrate.quad()` を使っても，同じ答えが出るようです。